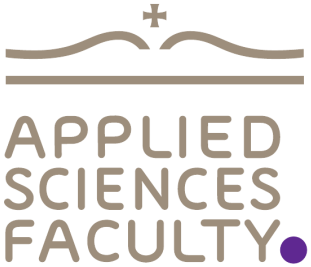
C++ — додаток. Бібліотеки





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- Бібліотека — об'єднання готових шматків коду, готових для повторного використання.
- В C++ це можуть бути функції, класи, константи, перерахування, глобальні змінні та змінні в просторах імен, тощо. Надалі говоритимемо саме про бібліотеки C та C++.
- Традиційно, оголошення всіх таких сутностей знаходиться в файлах заголовків (headers, *.h або *.hpp), реалізація — у об'єктних файлах, скомпільованих із відповідних *.c/*.cpp файлів.
- Характерною ознакою бібліотек є те, що ними можуть користуватися не пов'язані між собою програми (або слабо пов'язані частини дуже великих програм).
- Бібліотеки бувають двох видів — статичні та динамічні[1].
- Статичні бібліотеки вкомпільовуються безпосередньо в програму — статичне компонування (linking).
- Компонування з динамічними бібліотеками відбувається під час завантаження програми.

[1] Насправді, їх правильна назва: динамічні бібліотеки спільного використання (shared) — могли б існувати статичні бібліотеки спільного використання, однак у всіх популярних реалізаціях вони динамічні.

Статичні бібліотеки

- Статичні бібліотеки — всього лиш архів об'єктних файлів.
- В UNIX-подібних ОС (включаючи Linux, FreeBSD, тощо), для іменування бібліотек прийнято такий формат: lib**name**.a.
- Під Windows використовується **name**.lib.
- На далі розглядатимемо на прикладі GCC та Linux.
- ◇ Ідейно відмінності в Windows мінімальні, хоча є важливі нюанси.
- Всі використані засоби доступні і в Windows, за допомогою Mingw. Рекомендую скористатися дистрибутивом MSYS2[2]: `https://www.msys2.org/`.

[2] Менеджер пакетів – знайомий по Arch Linux pacman.

- Розглянемо програму із трьох .cpp файлів, яка була використана в лекції, присвяченій makefile. Будемо вважати main_file.cpp головною програмою, а функції з file1.cpp/file1.h і file2.cpp/file2.h бібліотечними.

Щоб створити бібліотеку:

- Слід скомпілювати всі об'єктні файли:

```
$ g++ file1.cpp -o file1.o -O3 -c
$ g++ file2.cpp -o file2.o -O3 -c
```

- Наступний крок — об'єднати ці файли в архів:

```
$ ar rcs libfunctions.a file1.o file2.o
```

- ar — програмка-архіватор, rcs — це опції[3], r — вставляти файли в архів, заміняти, якщо такі вже присутні, c — створювати архів, якщо його не існує, s — створювати індекс архіву (важлива, але технічна опція). Далі йде ім'я архіву та список об'єктних файлів.
- Користувачам бібліотеки постачаються файли file1.h, file2.h і libfunctions.a.

[3]Під Unix-подібними ОС опції передаються в формі "-o" або "–option", під Windows — "/o" або "/option". Однак, в деяких, особливо старих (хоча б за походженням) програмах опції передаються без спеціальних індикаторів опцій, наприклад, просто літерами, як в ar.

Використання бібліотек

- Компіляція файлів програми відбувається звичним чином.
- Під час компонування слід вказати, які бібліотеки використовувати:

```
g++ main_file.o -o prog -lfunctions -L.
```

- Ім'я вказується після опції -l: -lfunctions, при тому, для пошуку файлу бібліотеки, префікс lib і суфікс .a додаються автоматично.
- Лінкер проглядатиме бібліотечні файли зліва направо! Тому, якщо бібліотеки зв'язані між собою, порядок їх вказання важливий!
- Опція -L використовується, щоб показати, де шукати файли бібліотек. -L. — в поточній (".") директорії. Інший приклад: -L/usr/local/mylibrary/lib.
- Аналогічно, під час компіляції, шлях до файлів заголовків можна вказати за допомогою опції `-I`. Поточна директорія, зазвичай, включається автоматично (принаймні, для файлів, які включаються так: "file1.h"), хоча іноді доводиться писати `-I ...` Для інших можна робити так: `-I/usr/local/mylibrary/include`:

```
g++ main_file.cpp -o main_file.o -c -I/usr/local/mylibrary/include
```

Посилання

- Про деякі нюанси: `https://eli.thegreenplace.net/2013/07/09/library-order-in-static-linking`.

- Динамічні бібліотеки значно складніші — вони завантажуються під час виконання програми, компонування відбувається динамічно. Щоб цей процес відбувався, код бібліотеки повинен задовольняти ряд умов, і у ній повинна бути інформація, про те, які сутності вона надає клієнтам — які функції, класи, тощо, вона **експортує**.
- Компілюємо:

```
$ g++ file1.cpp -o file1.o -O3 -c -fPIC
file1.cpp:1:0: warning: -fPIC ignored for target (all
code is position independent) [enabled by default]
 #include <iostream>
 ^

$ g++ file2.cpp -o file2.o -O3 -c -fPIC
file2.cpp:1:0: warning: -fPIC ignored for target (all
code is position independent) [enabled by default]
 #include <iostream>
```

- ◇ Попередження можна ігнорувати, деталі зараз несуттєві.
- Опція -fPIC вказує генерувати код, який зможе виконуватися незалежно від того, де в пам'яті, за якою *лінійною* адресою, бібліотеку було завантажено.

- Компонування об'єктних файлів у бібліотеку для Linux:

  ```
  g++ -shared -fPIC -o libfunctions.so file1.o file2.o
  ```

  Для Linux, розширення ".so" – shared object. Для інших операційних систем, із використанням GCC, компіляція виглядатиме схоже. Хіба що розширення бібліотек відрізнятиметься – ".dll", dynamic link library (іноді dynamic loading library), для MS Windows, ".dylib" для MacOS.
- Опція -shared вказує генерувати динамічну (shared) бібліотеку.
- Решта опцій звичні.
- Компонування програми, яка використовує цю динамічну бібліотеку, можна робити так:

  ```
  g++ main_file.o -o prog libfunctions.so
  ```

  вказуючи файл бібліотеки безпосередньо, на рівні із об'єктними файлами.
- Альтернатива – зробити аналогічно до лінкування із статичною бібліотекою:

  ```
  g++ main_file.o -o prog -lfunctions
  ```

  Зазвичай, по замовчуванню GCC обере динамічну, якщо присутні обидві (libfunctions.so і libfunctions.a).

Пошук динамічних бібліотек – Linux

- Під Linux, пошук динамічних бібліотек здійснюється за шляхами, заданими наступними способами у такому порядку:

1. Шляхи rpath, вкомільовані у виконавчий файл[4]. Шляхи розділяються двокрапкою.
2. Шляхи, перераховані в змінній середовища **LD_LIBRARY_PATH**. Шляхи розділяються двокрапкою. Єдиний спосіб, доступний користувачу без прав root та потреби змінювати виконавчий файл.
3. Шляхи runpath, вкомпільовані у виконавчий файл. Шляхи розділяються двокрапкою.
4. У списку директорій із /etc/ld.so.conf, який неможливо змінити без прав root.

- В сам виконавчий файл може бути вкомпільовано rpath і або runpath – шлях для пошуку динамічних бібліотек цього файлу.

5. Може здійснюватися пошук в /lib, /usr/lib і ще декількох стандартних шляхах.

[4] Вважається застарілим, на користь runpath, так як останній може коригуватися LD_LIBRARY_PATH, а перший ні.

Пошук динамічних бібліотек – Linux: rpath та runpath

- Шляхи в rpath/runpath можуть бути як абсолютними, відносними відносно поточної директорії (".") та шлях відносно спеціального значення $ORIGIN – яке означає шлях до виконавчого файлу[5].
- З міркувань безпеки, для виконавчих файлів із бітами setuid/setguid – такими, що працюють із правами root, навіть коли запущені від імені користувача (приклад – passwd), LD_LIBRARY_PATH та rpath/runpath, як містять $ORIGIN, ігноруються.
- Задати runpath[6] можна за допомогою лінкера так:

```
gcc main.o /path/to/my/lib/libfiles.so -L. -o test \
                        -Wl,-rpath,/path/to/my/lib
```

  або:

```
gcc main.o ./libfiles.so -o test -Wl,-rpath,\$ORIGIN
```

  що означатиме – шукати в поточній директорії. \ перед $ обов'язковий, інакше shell вважатиме це підстановкою змінної середовища.

- Деякі подробиці – див. далі.

[5] Це не змінна середовища – просто таке ім'я.

[6] Для сучасних (після 2005) лінкерів – саме runpath, це не помилка. rpath вважається застарілим.

Пошук динамічних бібліотек – ldd/elfread/objdump

- ldd – скрипт, який за допомогою динамічного лоадера знаходить бібліотеки, що завантажуються:

```
ldd ./dynamic_lib_used
        linux-vdso.so.1 (0x00007fffd42d8000)
        libfiles.so => ./libfiles.so (0x00007f9328da0000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f9328b90000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f9328dc0000)
```

- Якщо не знайдено:

```
$ ./dynamic_lib_used
./dynamic_lib_used: error while loading shared libraries: libfiles.so: cannot open
 shared object file: No such file or directory
$ ldd ./dynamic_lib_used
        linux-vdso.so.1 (0x00007fffca14e000)
        libfiles.so => not found
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f76d29d0000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f76d2bfd000)
```

- Де шукало, можна побачити так:

```
$ LD_DEBUG=libs ./dynamic_lib_used
      5235:     find library=libfiles.so [0]; searching
      5235:      search cache=/etc/ld.so.cache
      5235:      search path=/lib/x86_64-linux-gnu/tls/haswell/x86_64: <skipped>
(system search path)
      5235:       trying file=/lib/x86_64-linux-gnu/tls/haswell/x86_64/libfiles.so
      5235:       trying file=/lib/x86_64-linux-gnu/tls/haswell/libfiles.so
      5235:       trying file=/lib/x86_64-linux-gnu/tls/x86_64/libfiles.so
<skipped>
```

Пошук динамічних бібліотек – ldd/elfread/objdump

- ldd показує динамічну ситуацію – що і як вдалося завантажити. Потрібні виконавчому файлу бібліотеки можна побачити за допомогою objdump чи readelf:

```
$ readelf -d dynamic_lib_used | grep NEEDED
 0x0000000000000001 (NEEDED)             Shared library: [libfiles.so]
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
$ objdump -x dynamic_lib_used | grep NEEDED
  NEEDED               libfiles.so
  NEEDED               libc.so.6
```

- ldd із прикладів вище показує також /lib64/ld-linux-x86-64.so.2 і linux-vdso.so.1. Перше – це, власне, динамічний лоадер, а друге – спеціальний трюк ядра, яким воно відображає деякі свої області пам'яті, як оптимізацію. Детальніше див. `https://man7.org/linux/man-pages/man7/vdso.7.html`, а ми трохи вивчатимемо на наступних курсах.

Керування runpath/rpath

- Для сучасних лінкерів (201x+), щоб задати **runpath**, потрібно передати опцію:

  `gcc <..skipped...> -Wl,-rpath,<path>`

  а щоб задати застарілий rpath:

  `gcc <..skipped...> -Wl,--disable-new-dtags,-rpath,<path>`
- Раніше було навпаки – -Wl,-rpath задавав rpath, а для runpath потрібно було:

  `gcc <..skipped...> -Wl,--enable-new-dtags,-rpath,<path>`
- Побачити вміст runpath/rpath можна так:

```
$ readelf ./dynamic_lib_used -d | grep path -i
 0x000000000000001d (RUNPATH)            Library runpath: [/path/to/lib]
readelf ./dynamic_lib_used_old -d | grep path -i
 0x000000000000000f (RPATH)              Library rpath: [/path/to/lib]
```

- Або за допомогою objdump:

```
$ objdump -x ./dynamic_lib_used_orig | grep path -i
  RUNPATH              $ORIGIN
$ objdump -x ./dynamic_lib_used | grep path -i
  RUNPATH              /path/to/lib
```

◇ Опція -i – –ignore-case для grep потрібна, що знайти і PATH, і path.

- **Увага!** rpath, вказаний для бібліотеки, заборонить завантажувати її із іншого шляху!

Керування runpath/rpath

- Змінити rpath/runpath можна за допомогою chrpath або patchelf.
- chrpath:

```
$ chrpath -l ./dynamic_lib_used    # Показати
./dynamic_lib_used: RUNPATH=/path/to/lib
$ chrpath -r \$ORIGIN ./dynamic_lib_used # Замінити
./dynamic_lib_used: RUNPATH=/path/to/lib
./dynamic_lib_used: new RUNPATH: $ORIGIN
$ chrpath -l ./dynamic_lib_used       # Перевіримо
./dynamic_lib_used: RUNPATH=$ORIGIN
$ readelf -d ./dynamic_lib_used |grep path -i # І ще так
 0x000000000000001d (RUNPATH)      Library runpath: [$ORIGIN]
$ chrpath -d ./dynamic_lib_used          # Видаляємо
$ chrpath -l ./dynamic_lib_used
./dynamic_lib_used: no rpath or runpath tag found.
```

Додавати поки не вміє. Подробиці див. https://linux.die.net/man/1/chrpath.

Керування runpath/rpath

- patchelf:

```
$ # Встановлюємо runpath
$ patchelf --set-rpath /path/to/lib ./dynamic_lib_use
$ # Саме rpath:
$ patchelf --force-rpath --set-rpath /path/to/lib ./dynamic_lib_us
$ patchelf --print-rpath ./dynamic_lib_used
/path/to/lib
$ patchelf --remove-rpath ./dynamic_lib_used
```

- Поміж інших цікавих опцій: ”–add-needed LIBRARY”, ”–replace-needed LIB_ORIG LIB_NEW”, ”–remove-needed LIBRARY”.
- А також:

```
$ patchelf --print-interpreter ./dynamic_lib_used
/lib64/ld-linux-x86-64.so.2
```

- Детальніше див. `https://github.com/NixOS/patchelf`.

Динамічні бібліотеки Linux – посилання

- `https://medium.com/@dtrugman/elf-loaders-libraries-and-executables-on-linux-e5cfce318f94`
- `https://amir.rachum.com/blog/2016/09/17/shared-libraries/`
- `https://en.wikipedia.org/wiki/Rpath`
- `https://gitlab.kitware.com/cmake/community/-/wikis/doc/cmake/RPATH-handling` – керування в CMake. Також містить опис пошуку бібліотек для MacOS: **Mac OS X and the RPATH**.

Пошук динамічних бібліотек – Windows

- Під Windows використовується все та ж **PATH**, що й для пошуку виконавчих файлів.
- ◇ Для зворотної сумісності також здійснюється пошук у директорії, де знаходиться виконавчий файл.
- ◇ Ці інструменти достатньо архаїчні. Сучасні Windows використовують SxS — Side-by-side Assembly та маніфести. Ціль – справитися із Dll hell та дозволити кожній програмі використовувати потрібну їй версію бібліотеки. Детальніше див., наприклад: `https://habr.com/ru/post/230505/`.

Статичні бінарники Linux

- Для GCC ключ -static компілює виконавчий файл виключно статично – без включення динамічних бібліотек.

  ```
  g++ -static <.............>
  ```

- Плюси і мінуси зрозумілі із плюсів та мінусів динамічних бібліотек.
- Потребує наявності відповідних статичних бібліотек.
- Під Windows все рівно лінкуватиме динамічно із системними бібліотеками ntdll.dll, KERNEL32.DLL, msvcrt.dll і т.д.
- -static-pie – PIE, але статичний.
- Для спеціальних задач див. також ключі -nostdlib, -nolibc, -nodefaultlibs, -nostartfiles.

Linux: soname

- Linux використовує поняття soname – ім'я бібліотеки, що зберігається у файлі .so.
- Його роль – задати назву бінарно сумісної версії динамічної бібліотеки.
- Справжнє ім'я файлу відрізнятиметься.
- Наприклад, для нашої бібліотеки, soname буде libfunctions.so.1 – перша версія, а зберігатиметься вона у файлі з іменем libfunctions.so.1.2.3, де 2 – номер підверсії, а 3 вказує – це третє виправлення до бібліотеки версії 1.2.
- Зазвичай, на файловій системі існуватиме символьний лінк із іменем, що співпадає з soname, на справжній файл із бібліотекою. Виконавчі файли, що використовують бібліотеку, посилатимуться на soname, а оновлення бібліотеки здійснюватиметься додаванням нової версії (наприклад, libfunctions.so.1.2.4) та перенаправленням символьного лінка.

```
g++ -shared -fPIC -Wl,-soname,libfunctions.so.1 \
    -o libfunctions.so.1.0.0 file1.o file2.o
```

- -Wl вказує, що передати безпосередньо лінкеру. Йому передасться опція "-soname libfunctions.so.1" – кома позначає пробіл.
- Цей механізм – підхід Linux до вирішення проблеми, яку під Windows називають "Dll hell".
- Детальніше див. `http://tldp.org/HOWTO/Program-Library-HOWTO/shared-libraries.html`.

Файли імпорту динамічних бібліотек

- Під **Windows** іноді зручно мати відповідний .a-файл, як його в цьому випадку називають, бібліотеку імпорту[7]. Отримується він так:

```
g++ -shared \
-Wl,--out-implib,libfunctions.a\
  -o libfunctions.dll file1.o file2.o
```

- Компіляція тоді виглядатиме звично:

```
g++ main_file.o -o prog -L. -lfunctions
```

- Старіші версії GCC під Linux теж потребували файл імпорту. Часто можна було зустріти: libname.a – статична бібліотека, libname.so – динамічна бібліотека та libname.so.a – її файл імпорту. Зараз потреби у .so.a-файлах немає.

[7] Документація стверджує, що "*This option is specific to the i386 PE targeted port of the linker*", але і MinGW-64 із 64-бітною цільовою архітектурою (target) теж розуміє –out-implib.

Windows

- Windows передбачає засоби явного керування, що експортувати із бібліотеки чи що імпортувати.
- Також, існують засоби керування угодами про виклики, які використовувати. Вони **НЕ НАЛЕЖАТЬ** до стандартних засобів C/C++, однак підтримуються багатьма компіляторами для Windows, зокрема — GCC/MinGW:

```
__stdcall void print_text(const char *s);
```

- Підтримуються `__cdecl`, `__stdcall` та інші. Деталі — див. документацію, наприклад `http://msdn.microsoft.com/en-us/library/zxk0tw93(VS.80).aspx` і `https://gcc.gnu.org/onlinedocs/gcc-4.8.3/gcc/Function-Attributes.html#Function-Attributes`.
- Щоб вказати, що функція експортується, (компілюючи код бібліотеки) пишемо так:

```
void __stdcall __declspec(dllexport) print_text(const char *s);
```

- Для імпортування, (компілюючи код програми):

```
void __stdcall __declspec(dllimport) print_text(const char *s);
```

- Оскільки мати два майже ідентичних заголовки, для побудови програми і для побудови самої бібліотеки дуже незручно, зазвичай роблять так:

```
#ifdef BUILDING_DLL
#define IMPORT_EXPORT __declspec(dllexport)
#else
#define IMPORT_EXPORT __declspec(dllimport)
#endif
      .....................
void __stdcall IMPORT_EXPORT print_text(const char *s);
```

- Тоді, компілюючи файли DLL, робимо так:

  g++ −D BUILDING_DLL ....

  а компілюючи саму програму, того не робимо, не означаємо відповідний макрос.
- Те ж саме можна зробити, використавши .DEF-файл, в якому описати, що експортуватиметься. Детальніше ми поки не розглядатимемо. Див., наприклад: `http://www.ni.com/white-paper/3056/en/` — "Building a DLL with Visual C++" (особливо частина третя, "Step 3: Exporting Symbols").

- Альтернативою до використання `__declspec(dllimport)` та `__declspec(dllexport)` є створення файлу опису модуля – DEF-файлу.
- Цей файл містить список сутностей, що експортуються бібліотекою. Детальніше тут ми його не розглядатимемо, за подробицями див. `https://github.com/MicrosoftDocs/cpp-docs/blob/master/docs/build/exporting-from-a-dll-using-def-files.md`.
- ◇ Приклад звідти:

```
LIBRARY   BTREE
EXPORTS
   Insert   @1
   Delete   @2
   Member   @3
   Min   @4
```

де BTREE – назва бібліотеки, Insert, Delete, Member та Min – експортовані функції, @x – це так звані ординали, їх вказувати не обов'язково, а LIBRARY і EXPORTS – ключові слова.

- Як використати def-файли за допомогою gcc-mingw: `http://jetsnote.blogspot.com/2016/04/dll-in-mingw.html` – але це дещо плутана дорога.

- Однак, іноді DEF-файл потрібен, щоб використовувати бібліотеку з Visual Studio. GCC можна попросити створити його, лінкуючи бібліотеку:

```
g++ -shared \
  -Wl,--out-implib,libfunctions.a,\
  --output-def,functions.def\
  -o libfunctions.dll file1.o file2.o
```

- Вона згенерує наступний файл `functions.def`:

```
EXPORTS
    function_from_file1 @1
    function_from_file2 @2
```

якщо код був на C.

- Однак, для коду на C++ буде отримано щось таке:

```
EXPORTS
    _Z19function_from_file1v @1
    _Z19function_from_file2v @2
```

- Це **заплутані (mangled)** імена. Вони використовуються C++ для підтримки перевантаження функцій.
- Про нього – трохи далі.

Компіляція з використанням MSVC – command line

- Компілятор Visual studio потребує багато змінних середовища і доволі складної його, середовища, підготовки.
- З компілятором постачається скрипт, .bat файл, який виконує цю підготовку.
- В пошуку/меню Start воно називається якось так: "x64 Native Tools Command Prompt for VS 2019".
- При запуску буде виконано команду:

```
%comspec% /k "C:\Program Files (x86)\Microsoft Visual Studio\
   2019\Community\VC\Auxiliary\Build\vcvars64.bat"
```

- Там доступні компілятор cl.exe, лінкер link.exe, "бібліотекар" lib.exe, асемблер ml64.exe тощо.
- Драйвер cl.exe достатньо потужний, як і gcc – вміє багато всього.
- ◇ Приклади нижче – аж ніяк не вичерпні, лише для ілюстрації.
- Ремарка: для зворотної сумісності Windows робить вигляд, що із кожним диском пов'язано свою поточну директорію. Для переключення диску використовуйте X: де X – літера диску.

```
C:\Users\indrekis>cd d:\Temp\Students\video\code\sample1
C:\Users\indrekis>d:
d:\Temp\Students\video\code\sample1>
```

- Історично так склалося[8], що опції в утилітах Windows визначаються не символом "-" а "/".
- Опції різних інструментів MSVC менш узгоджені, між різними інструментами, ніж в Linux.
- Найпростіший варіант:

```
d:\Temp\Students\video\code\sample1>cl *.c /Fe:test.exe
Microsoft (R) C/C++ Optimizing Compiler Version 19.26.28806 for x64
Copyright (C) Microsoft Corporation.  All rights reserved.

critical_function.c
function_with_complex_args.c
important_function.c
main.c
Generating Code...
Microsoft (R) Incremental Linker Version 14.26.28806.0
Copyright (C) Microsoft Corporation.  All rights reserved.

/out:test.exe
critical_function.obj
function_with_complex_args.obj
important_function.obj
main.obj
```

- Опція /nologo зробить вивід значно лаконічнішим.

[8] http://www.os2museum.com/wp/why-does-windows-really-use-backslash-as-path-separator/

Компіляція з використанням MSVC – статична бібліотека

- Створювати статичну бібліотеку може як lib.exe, так і link.exe.
- Опція /Fo задає ім'я об'єктного файлу для cl.exe. Але якщо передати декілька c-файлів, він використає імена вхідних файлів за основу для імен об'єктних:

```
[PATH]>cl /nologo /c *.c
critical_function.c
function_with_complex_args.c
important_function.c
main.c
Generating Code...
[PATH]>lib /OUT:functions.lib critical_function.obj
  function_with_complex_args.obj important_function.obj
  /nologo
# або link /lib <.......>
[PATH]>cl /Fe:test.exe main.obj /link functions.lib /nologo
```

Компіляція з використанням MSVC – динамічна бібліотека 1

- Спершу скористаємося .DEF-файлом:

```
LIBRARY functions
EXPORTS
    critical_function @1
    function_with_complex_args @2
    important_function @3
```

-
```
[PATH]>cl /nologo /c *.c
[PATH]>link /dll /OUT:functions.dll /DEF:functions.def
 critical_function.obj function_with_complex_args.obj
 important_function.obj /nologo
   Creating library functions.lib and object functions.exp
```

- При створенні динамічної бібліотеки, окрім functions.dll створюється також functions.lib – бібліотеку імпорту та functions.exp – бібліотеку експорту[9].
- Чомусь, якщо поставити /Fe:test.exe в кінці – тут проігнорує, але в попередніх прикладах – враховувало.

[9] Детальніше див. https://docs.microsoft.com/ru-ru/cpp/build/reference/dot-exp-files-as-linker-input?view=vs-2019.

Компіляція з використанням MSVC – динамічна бібліотека 2

- Спершу скористаємося .DEF-файлом:

```
LIBRARY functions
EXPORTS
    critical_function @1
    function_with_complex_args @2
    important_function @3
```

-
```
[PATH]>cl /nologo /c critical_function.c
     function_with_complex_args.c
     important_function.c /DBUILDING_DLL
[PATH]>cl /nologo /c main.c
[PATH]>link /dll /OUT:functions.dll /IMPLIB:functions.lib
 critical_function.obj function_with_complex_args.obj
 important_function.obj /nologo
   Creating library functions.lib and object functions.exp
[PATH]>cl /Fe:test2.exe main.obj /link functions.lib /nologo
```

- Якщо functions.lib не створюється – щось наплутано і бібліотека нічого не експортує.

Функціонування динамічних бібліотек Windows – посилання

- "The Import Address Table is now write-protected, and what that means for rogue patching".
- "What is DLL import hinting?".
- "What is DLL import binding?".
- "Why does the Windows Portable Executable (PE) format have separate tables for import names and import addresses?, part 1".
- "Why does the Windows Portable Executable (PE) format have separate tables for import names and import addresses?, part 2".
- "How important is it nowadays to ensure that all my DLLs have non-conflicting base addresses?".
- "If relocated DLLs cannot share pages, then doesn't ASLR cause all pages to be non-shared?".
- "Why should I even bother to use DLL's in my system?" – дещо застаріла, завдяки ASLR механізму вище, але все ще цікава.
- "Why are the module timestamps in Windows 10 so nonsensical?"
- "How can I view the list of symbols available in a library?"

Заплутування імен – name mangling та extern "C"

- Різні C++ компілятори заплутують імена по різному.
- Так як різні компілятори C++[10] між собою на рівні бінарного коду[11] не сумісні, це навіть добре.
- Мінусом є те, що всі бібліотеки, які входять в програму, потрібно компілювати одним компілятором. Далеко не завжди це можливо.
- Альтернатива – всі компілятори розуміють бінарний інтерфейс (ABI) коду C своїх платформ – як спільний знаменник. Зокрема, всі вони зможуть викликати бібліотеки C – одна із вимог, завдяки дотриманню якої C++ і набуло популярності.
  - ◇ Тому інтерфейс в стилі C може бути посередником між бінарними модулями, скомпільованими різними компіляторами.
- Однак, коли C++ компілятор аналізуватиме файл заголовків C та вставлятиме виклики функцій, він імена теж "заплутає".
- Щоб попросити його не робити це, потрібно використати модифікатор extern "C". Імена таких функцій будуть сумісні із прийнятими в C. На жаль, перевантаження функцій при цьому не працюватиме.
- Певні несумісності між компіляторами можливі і в цьому випадку, але, зазвичай, все працюватиме.

[10] Іноді, наприклад, MSVC до 2017, навіть різні версії компіляторів

[11] Скоро ми це будемо називати ABI.

```
#ifndef FILE1_HEADER
#define FILE1_HEADER

#ifdef __cplusplus
extern "C" {
#endif

void function_from_file1();

#ifdef __cplusplus
}
#endif

#endif // FILE1_HEADER
```

- Решта файлів модифікуємо аналогічно.
- Компілюємо звично:

```
gcc file1.c -o file1.o -O3 -c -fPIC  -I.
file1.c:1:0: warning: -fPIC ignored for target (all code is
 position independent) [enabled by default]
 #include <stdio.h>
 ^
```

- Після компонування бачимо в DEF-файлі:

```
EXPORTS
    function_from_file1 @1
    function_from_file2 @2
```

- Взагалі, подивитися, які символи бібліотека чи об'єктний файл експортує чи імпортує, можна за допомогою утиліти nm:

```
$ nm file1.o #C-file!
0000000000000000 b .bss
0000000000000000 d .data
0000000000000000 p .pdata
0000000000000000 r .rdata
0000000000000000 r .rdata$zzz
0000000000000000 t .text
0000000000000000 r .xdata
0000000000000000 T function_from_file1
                 U puts
```

- nm – утиліта, що вміє показувати символи із об'єктного файлу.
- T значить міститься в цьому файлі, U — повинен бути розв'язаним під час компонування.
- puts – наслідок оптимізації. Компілятор має право замінити printf() із одним аргументом-стрічкою на puts().

```
$ nm file1.o # <=== C++-file!
0000000000000000 b .bss
0000000000000000 d .ctors
0000000000000000 d .data
0000000000000000 p .pdata
0000000000000000 p .pdata.startup
0000000000000000 r .rdata
0000000000000000 r .rdata$zzz
0000000000000000 t .text
0000000000000000 t .text.startup
0000000000000000 r .xdata
0000000000000000 r .xdata.startup
0000000000000000 t __tcf_0
0000000000000000 t _GLOBAL__sub_I_file1.cpp
0000000000000010 T _Z19function_from_file1v
                 U _ZNKSt5ctypeIcE13_M_widen_initEv
                 U _ZNSo3putEc
                 U _ZNSo5flushEv
                 U _ZNSt8ios_base4InitC1Ev
                 U _ZNSt8ios_base4InitD1Ev
                 U _ZSt16__ostream_insertIcSt11char_traitsIcEERSt13ba
                 U _ZSt16__throw_bad_castv
                 U _ZSt4cout
```

```
$ nm file1.o -C # <=== C++-file, ввімкнено demangling
0000000000000000 b .bss
0000000000000000 d .ctors
0000000000000000 d .data
0000000000000000 p .pdata
0000000000000000 p .pdata.startup
0000000000000000 r .rdata
0000000000000000 r .rdata$zzz
0000000000000000 t .text
0000000000000000 t .text.startup
0000000000000000 r .xdata
0000000000000000 r .xdata.startup
0000000000000000 t __tcf_0
0000000000000000 t _GLOBAL__sub_I_file1.cpp
0000000000000010 T function_from_file1()
                 U std::ctype<char>::_M_widen_init() const
                 U std::ostream::put(char)
                 U std::ostream::flush()
                 U std::ios_base::Init::Init()
                 U std::ios_base::Init::~Init()
                 U std::basic_ostream<char, std::char_traits<char> >&
                 U std::__throw_bad_cast()
                 U std::cout
```

Відкладене завантаження динамічних бібліотек

- Можна також завантажувати бібліотеки під час виконання програми. Таким чином реалізовуються, наприклад, плагіни.
- Під UNIX-подібними ОС для цього служать функції із файлу <dlfcn.h> (відповідна бібліотека — libdl.a) :

```
void * dlopen(const char *filename, int flag );
char * dlerror (void);
void *dlsym(void *handle, const char *symbol);
int dlclose (void *handle);
```

Детальніше див., наприклад, `http://linux.die.net/man/3/dlerror` і `http://tldp.org/HOWTO/Program-Library-HOWTO/dl-libraries.html`

- Аналогічні засоби існують і для Windows. Це функції LoadLibrary і LoadLibraryEx. Детальніше див. `http://msdn.microsoft.com/en-us/library/windows/desktop/ms685090(v=vs.85).aspx` і `http://msdn.microsoft.com/en-us/library/ms810279.aspx`.

Приклад (взято з http://tldp.org/HOWTO/Program-Library-HOWTO/dl-libraries.html):

```
#include <stdlib.h>
#include <stdio.h>
#include <dlfcn.h>

int main(int argc, char **argv) {
    void *handle;          char *error;
    double (*cosine)(double);

    handle = dlopen ("/lib/libm.so.6", RTLD_LAZY);
    if (!handle) {
        fputs (dlerror (), stderr);    exit (1);
    }

    cosine = dlsym(handle, "cos");
    if ((error = dlerror ()) != NULL) {
        fputs(error, stderr);          exit (1);
    }

    printf ("%f\n", (*cosine)(2.0));
    dlclose (handle);
}
```

- http://www.adp-gmbh.ch/cpp/gcc/create_lib.html — проста розповідь, як створювати бібліотеки.
- http://www.mingw.org/wiki/sampleDLL — побудова DLL, з допомогою MinGW.
- http://www.mingw.org/wiki/MSVC_and_MinGW_DLLs — нюанси використання бібліотек, створених MinGW з MSVC і навпаки.
- http://www.ni.com/white-paper/3056/en/ — "Building a DLL with Visual C++"
- http://linux.die.net/man/3/dlerror і http://tldp.org/HOWTO/Program-Library-HOWTO/dl-libraries.html — завантаження динамічних бібліотек на льоту (UNIX-подібні ОС).
- http://msdn.microsoft.com/en-us/library/windows/desktop/ms685090(v=vs.85).aspx — "Run-Time Dynamic Linking" для MSVC.
- http://msdn.microsoft.com/en-us/library/ms810279.aspx — "DLLs the Dynamic Way".
- Короткий огляд кросплатформового створення динамічних бібліотек: https://gernotklingler.com/blog/creating-using-shared-libraries-different-compilers-different-ope

Есеї та статті на тему

- `https://devblogs.microsoft.com/oldnewthing/20230922-00/?p=108813` – "When I try to call an exported function, the target crashes when it tries to call any Windows function", про особливості завантаження виконавчих файлів як DLL.
- `https://lucisqr.substack.com/p/shared-lto-plt-friends-or-foes` – про ціну, з точки зору продуктивності, використання динамічних бібліотек, наявності PLT тощо.
- `https://blogsystem5.substack.com/p/glibc-versions-runtime` – есей про динамічне зв'язування, дещо наївний, але цікавий.
- "Understanding the classical model for linking, groundwork: The algorithm"